Лабораторна робота №2

Породжувальні шаблони

Мета роботи: навчитися реалізовувати породжувальні шаблони проєктування.

Термін виконання: 2 год (1 заняття)

Зміст роботи

Завдання №1

1. Напишіть систему класів для реалізації функціоналу створення різних типів підписок для відео провайдера.

2. Кожна з підписок повинна мати щомісячну плату, мінімальний період підписки та список каналів й інших можливостей.

3. Види підписок: DomesticSubscription, EducationalSubscription, PremiumSubscription.

4. Придбати (тобто створити) підписку можна за допомогою трьох різних класів: WebSite, MobileApp, ManagerCall, кожен з них має реалізувати свою логіку створення підписок.

Лістинг програми:

```
using System;
using System.Collections.Generic;

#region Subscriptions

abstract class Subscription
{
    public decimal MonthlyFee { get; protected set; }
    public int MinimumPeriodMonths { get; protected set; }
    public List<string> Channels { get; protected set; }

    public void DisplayInfo()
    {
        Console.WriteLine($"Monthly fee: {MonthlyFee} UAH");
        Console.WriteLine($"Minimum period: {MinimumPeriodMonths} months");
        Console.WriteLine("Included channels:");

        foreach (var channel in Channels)
        {
            Console.WriteLine($" • {channel}");
        }
        Console.WriteLine();
    }
}
```

```
class DomesticSubscription : Subscription
{
    public DomesticSubscription()
    {
        MonthlyFee = 150;
        MinimumPeriodMonths = 3;

        Channels = new List<string>
        {
            "News",
            "Movies",
            "Entertainment"
        };
    }
}

class EducationalSubscription : Subscription
{
    public EducationalSubscription()
    {
        MonthlyFee = 90;
        MinimumPeriodMonths = 6;

        Channels = new List<string>
        {
            "Education",
            "Science",
            "Documentaries"
        };
    }
}

class PremiumSubscription : Subscription
{
    public PremiumSubscription()
    {
        MonthlyFee = 300;
        MinimumPeriodMonths = 1;

        Channels = new List<string>
        {
            "HD Movies",
            "HD Sports",
            "Music",
            "Ad-free experience"
        };
    }
}

#endregion

#region Factory

enum SubscriptionType
{
    Domestic,
    Educational,
    Premium
}

abstract class SubscriptionProvider
{
    public abstract Subscription CreateSubscription(SubscriptionType type);
}
```

```
class WebSite : SubscriptionProvider
{
    public override Subscription CreateSubscription(SubscriptionType type)
    {
        Console.WriteLine("Creating subscription via website...");

        return type switch
        {
            SubscriptionType.Domestic => new DomesticSubscription(),
            SubscriptionType.Educational => new EducationalSubscription(),
            SubscriptionType.Premium => new PremiumSubscription(),
            _ => throw new ArgumentException("Invalid subscription type")
        };
    }
}

class MobileApp : SubscriptionProvider
{
    public override Subscription CreateSubscription(SubscriptionType type)
    {
        Console.WriteLine("Creating subscription via mobile app...");

        return type switch
        {
            SubscriptionType.Domestic => new DomesticSubscription(),
            SubscriptionType.Educational => new EducationalSubscription(),
            SubscriptionType.Premium => new PremiumSubscription(),
            _ => throw new ArgumentException("Invalid subscription type")
        };
    }
}

class ManagerCall : SubscriptionProvider
{
    public override Subscription CreateSubscription(SubscriptionType type)
    {
        Console.WriteLine("Creating subscription via manager call...");

        return type switch
        {
            SubscriptionType.Domestic => new DomesticSubscription(),
            SubscriptionType.Educational => new EducationalSubscription(),
            SubscriptionType.Premium => new PremiumSubscription(),
            _ => throw new ArgumentException("Invalid subscription type")
        };
    }
}

#endregion

class Program
{
    static void Main()
    {
        SubscriptionProvider website = new WebSite();
        SubscriptionProvider mobileApp = new MobileApp();
        SubscriptionProvider manager = new ManagerCall();

        Subscription domestic = website.CreateSubscription(SubscriptionType.Domestic);
        domestic.DisplayInfo();

        Subscription educational =
mobileApp.CreateSubscription(SubscriptionType.Educational);
        educational.DisplayInfo();
```

```
        Subscription premium = manager.CreateSubscription(SubscriptionType.Premium);
        premium.DisplayInfo();

        Console.ReadKey();
    }
}
```

5. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Результат виконання програми:

```
C:\Users\Masha\source\repos\kpz_lab2\kpz_lab2\bin\Debug\net8.0\kpz_lab2.exe
Creating subscription via website...
Monthly fee: 150 UAH
Minimum period: 3 months
Included channels:
  News
  Movies
  Entertainment

Creating subscription via mobile app...
Monthly fee: 90 UAH
Minimum period: 6 months
Included channels:
  Education
  Science
  Documentaries

Creating subscription via manager call...
Monthly fee: 300 UAH
Minimum period: 1 months
Included channels:
  HD Movies
  HD Sports
  Music
  Ad-free experience
```

Рис.1. Результат виконання програми.

6. Підготуйте діаграму створених у програмі класів та інтерфейсів за допомогою https://app.diagrams.net/, експортуйте та завантажте її до репозиторія.

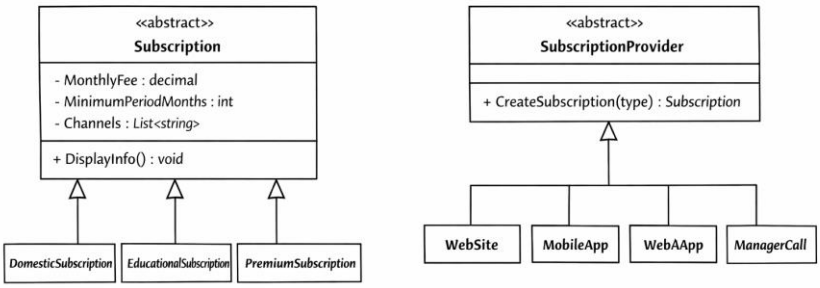

Рис.2. Діаграма створених у програмі класів.

Завдання №2

1. Створіть фабрику виробництва техніки.

2. На фабриці мають створюватися різні девайси (наприклад, Laptop, Netbook, EBook, Smartphone) для різних брендів (IProne, Kiaomi, Balaxy).

Лістинг програми:

```
using System;

#region Devices

interface ILaptop
{
    string GetModel();
}

interface INetbook
{
    string GetModel();
}

interface IEBook
{
    string GetModel();
}

interface ISmartphone
{
    string GetModel();
}

#endregion

#region Brand implementations


class IProneLaptop : ILaptop
{
    public string GetModel() => "IProne Laptop Pro";
}

class IProneNetbook : INetbook
{
    public string GetModel() => "IProne Netbook Air";
}

class IProneEBook : IEBook
{
    public string GetModel() => "IProne E-Book Reader";
}

class IProneSmartphone : ISmartphone
{
    public string GetModel() => "IProne Phone X";
}



class KiaomiLaptop : ILaptop
{
```

```
    public string GetModel() => "Kiaomi Notebook 15";
}

class KiaomiNetbook : INetbook
{
    public string GetModel() => "Kiaomi Netbook Mini";
}

class KiaomiEBook : IEBook
{
    public string GetModel() => "Kiaomi Reader";
}

class KiaomiSmartphone : ISmartphone
{
    public string GetModel() => "Kiaomi Smart 12";
}



class BalaxyLaptop : ILaptop
{
    public string GetModel() => "Balaxy Book Ultra";
}

class BalaxyNetbook : INetbook
{
    public string GetModel() => "Balaxy Netbook Go";
}

class BalaxyEBook : IEBook
{
    public string GetModel() => "Balaxy Reader Plus";
}

class BalaxySmartphone : ISmartphone
{
    public string GetModel() => "Balaxy S Pro";
}

#endregion

#region Abstract Factory

interface IDeviceFactory
{
    ILaptop CreateLaptop();
    INetbook CreateNetbook();
    IEBook CreateEBook();
    ISmartphone CreateSmartphone();
}

#endregion

#region Concrete factories

class IProneFactory : IDeviceFactory
{
    public ILaptop CreateLaptop() => new IProneLaptop();
    public INetbook CreateNetbook() => new IProneNetbook();
    public IEBook CreateEBook() => new IProneEBook();
    public ISmartphone CreateSmartphone() => new IProneSmartphone();
}

class KiaomiFactory : IDeviceFactory
```

```
{
    public ILaptop CreateLaptop() => new KiaomiLaptop();
    public INetbook CreateNetbook() => new KiaomiNetbook();
    public IEBook CreateEBook() => new KiaomiEBook();
    public ISmartphone CreateSmartphone() => new KiaomiSmartphone();
}

class BalaxyFactory : IDeviceFactory
{
    public ILaptop CreateLaptop() => new BalaxyLaptop();
    public INetbook CreateNetbook() => new BalaxyNetbook();
    public IEBook CreateEBook() => new BalaxyEBook();
    public ISmartphone CreateSmartphone() => new BalaxySmartphone();
}

#endregion

class Program
{
    static void ShowDevices(IDeviceFactory factory, string brand)
    {
        Console.WriteLine($"\n--- {brand} devices ---");

        Console.WriteLine(factory.CreateLaptop().GetModel());
        Console.WriteLine(factory.CreateNetbook().GetModel());
        Console.WriteLine(factory.CreateEBook().GetModel());
        Console.WriteLine(factory.CreateSmartphone().GetModel());
    }

    static void Main()
    {
        IDeviceFactory iprone = new IProneFactory();
        IDeviceFactory kiaomi = new KiaomiFactory();
        IDeviceFactory balaxy = new BalaxyFactory();

        ShowDevices(iprone, "IProne");
        ShowDevices(kiaomi, "Kiaomi");
        ShowDevices(balaxy, "Balaxy");

        Console.ReadKey();
    }
}
```

3. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Результат виконання програми:

```
C:\Users\Masha\source\repos\kpz_lab2\kpz_lab2\bin\Debug\net8.0\kpz_lab2.exe

--- IProne devices ---
IProne Laptop Pro
IProne Netbook Air
IProne E-Book Reader
IProne Phone X

--- Kiaomi devices ---
Kiaomi Notebook 15
Kiaomi Netbook Mini
Kiaomi Reader
Kiaomi Smart 12

--- Balaxy devices ---
Balaxy Book Ultra
Balaxy Netbook Go
Balaxy Reader Plus
Balaxy S Pro
```

Рис.3. Результат виконання програми.

4. Підготуйте діаграму створених у програмі класів та інтерфейсів за допомогою https://app.diagrams.net/, експортуйте та завантажте її до репозиторія.

Результат виконання програми:

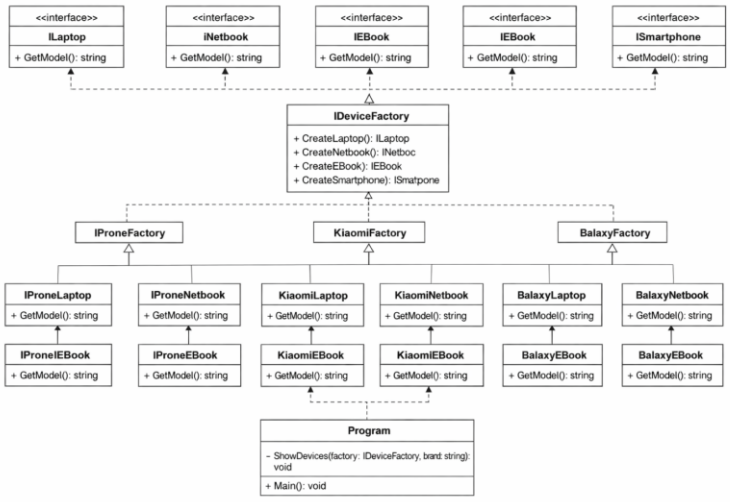

Рис.4. Діаграма створених у програмі класів.

Завдання №3

Одинак.

1. Створіть клас Authenticator таким чином, щоб бути впевненим, що цей клас може створити лише один екземпляр, незалежно від кількості потоків і класів, що його наслідують.

Лістинг програми:

```
using System;
```

```
#region Singleton: Authenticator

public class Authenticator
{
    private static Authenticator? _instance;
    private static readonly object _lock = new object();

    protected Authenticator()
    {
        Console.WriteLine("Authenticator instance created.");
    }


    public static Authenticator Instance
    {
        get
        {
            if (_instance == null)
            {
                lock (_lock)
                {
                    if (_instance == null)
                    {
                        _instance = new Authenticator();
                    }
                }
            }
            return _instance;
        }
    }

    public void Authenticate(string username, string password)
    {
        Console.WriteLine($"Authenticating {username}...");

    }
}

#endregion

#region Наслідування приклад

public class AdvancedAuthenticator : Authenticator
{
    public void AdvancedAuthenticate(string username)
    {
        Console.WriteLine($"Advanced authentication for {username}.");
    }
}

#endregion

class Program
{
    static void Main(string[] args)
    {

        var auth1 = Authenticator.Instance;
        var auth2 = Authenticator.Instance;

        Console.WriteLine($"auth1 == auth2: {auth1 == auth2}");

        auth1.Authenticate("user1", "pass1");
```

```
        var advAuth = new AdvancedAuthenticator();
        advAuth.AdvancedAuthenticate("user2");

        Console.WriteLine("Program finished.");
    }
}
```

2. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Результат виконання програми:

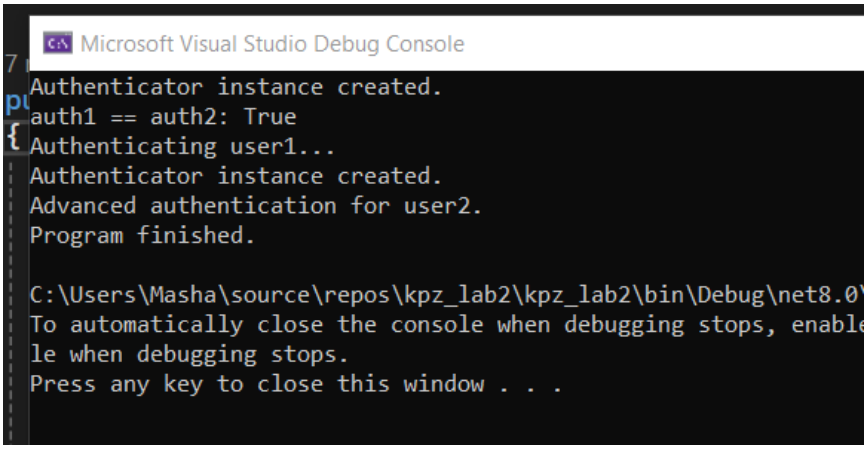

Рис.5. Результат виконання програми.

Завдання №4

1. Створіть клас Virus. Він повинен містити вагу, вік, ім'я, вид і масив дітей, екземплярів Virus.

2. Створіть екземпляри для цілого "сімейства" вірусів (мінімум три покоління).

3. За допомогою шаблону Прототип реалізуйте можливість клонування наявних вірусів.

4. При клонуванні віруса-батька повинні клонуватися всі його діти.

Лістинг програми:

```
using System;
using System.Collections.Generic;

#region Virus class with Prototype

public class Virus
{
    public string Name { get; set; }
    public string Species { get; set; }
    public int Age { get; set; }
    public double Weight { get; set; }
```

```
    public List<Virus> Children { get; set; } = new List<Virus>();

    public Virus(string name, string species, int age, double weight)
    {
        Name = name;
        Species = species;
        Age = age;
        Weight = weight;
    }

    public void AddChild(Virus child)
    {
        Children.Add(child);
    }

    public Virus Clone()
    {
        Virus clone = new Virus(Name, Species, Age, Weight);

        foreach (var child in Children)
        {
            clone.AddChild(child.Clone());
        }

        return clone;
    }

    public void Print(string indent = "")
    {
        Console.WriteLine($"{indent}- {Name} ({Species}), Age: {Age}, Weight: {Weight}");
        foreach (var child in Children)
        {
            child.Print(indent + "  ");
        }
    }
}

#endregion

class Program
{
    static void Main(string[] args)
    {

        Virus parent = new Virus("Alpha", "Corona", 5, 1.2);
        Virus child1 = new Virus("Beta", "Corona", 2, 0.8);
        Virus child2 = new Virus("Gamma", "Corona", 1, 0.5);
        Virus grandChild = new Virus("Delta", "Corona", 0, 0.2);

        child1.AddChild(grandChild);
        parent.AddChild(child1);
        parent.AddChild(child2);

        Console.WriteLine("Original Virus Family:");
        parent.Print();

        Virus clonedParent = parent.Clone();

        Console.WriteLine("\nCloned Virus Family:");
        clonedParent.Print();

        Console.WriteLine($"\nparent == clonedParent: {parent == clonedParent}");
        Console.WriteLine($"parent.Children[0] == clonedParent.Children[0]:
{parent.Children[0] == clonedParent.Children[0]}");
```

```
    }
}
```

5. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Результат виконання програми:

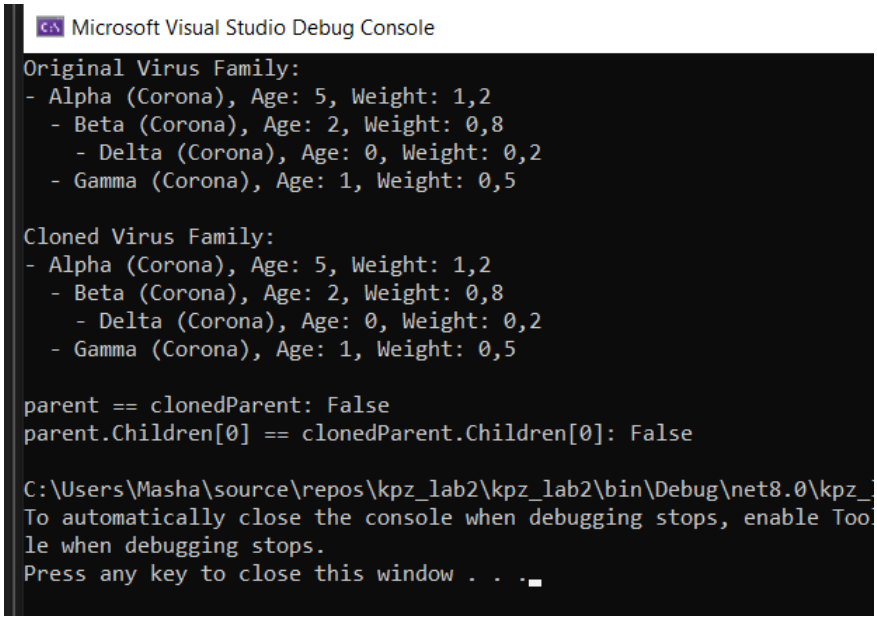

Рис.6. Результат виконання програми.

Завдання №5

Будівельник.

1. Створіть клас HeroBuilder, який буде створювати персонажа гри, поступово додаючи до нього різні ознаки, наприклад зріст, статуру, колір волосся, очей, одяг, інвентар тощо (можете включити фантазію).

2. Створіть клас EnemyBuilder, який буде реалізовувати єдиний інтерфейс з HeroBuilder. Відмінністю між ними можуть бути спеціальні методи для творення добра або зла, а також списки добрих і злих справ відповідно.

3. За допомогою свого білдера і класу-директора створіть героя (або героїню) своєї мрії ☺, а також свого найзапеклішого ворога.

4. Зверніть увагу, що Ваші білдери повинні реалізовувати текучий интерфейс (fluent interface).

Лістинг програми:

```
using System;
using System.Collections.Generic;

#region Builder Interface

public interface ICharacterBuilder<TBuilder>
{
    TBuilder SetHeight(int height);
    TBuilder SetBuild(string build);
    TBuilder SetHairColor(string hairColor);
    TBuilder SetEyeColor(string eyeColor);
    TBuilder SetClothes(string clothes);
    TBuilder AddInventory(string item);
    Character Build();
}

#endregion

#region Hero Builder

public class HeroBuilder : ICharacterBuilder<HeroBuilder>
{
    private Character _hero = new Character();

    public HeroBuilder SetHeight(int height) { _hero.Height = height; return this; }
    public HeroBuilder SetBuild(string build) { _hero.Build = build; return this; }
    public HeroBuilder SetHairColor(string hairColor) { _hero.HairColor = hairColor;
return this; }
    public HeroBuilder SetEyeColor(string eyeColor) { _hero.EyeColor = eyeColor; return
this; }
    public HeroBuilder SetClothes(string clothes) { _hero.Clothes = clothes; return this;
}
    public HeroBuilder AddInventory(string item) { _hero.Inventory.Add(item); return
this; }

    public HeroBuilder AddGoodDeed(string deed) { _hero.GoodDeeds.Add(deed); return this;
}

    public Character Build() { return _hero; }
}

#endregion

#region Enemy Builder

public class EnemyBuilder : ICharacterBuilder<EnemyBuilder>
{
    private Character _enemy = new Character();

    public EnemyBuilder SetHeight(int height) { _enemy.Height = height; return this; }
    public EnemyBuilder SetBuild(string build) { _enemy.Build = build; return this; }
    public EnemyBuilder SetHairColor(string hairColor) { _enemy.HairColor = hairColor;
return this; }
    public EnemyBuilder SetEyeColor(string eyeColor) { _enemy.EyeColor = eyeColor; return
this; }
    public EnemyBuilder SetClothes(string clothes) { _enemy.Clothes = clothes; return
this; }
    public EnemyBuilder AddInventory(string item) { _enemy.Inventory.Add(item); return
this; }

    public EnemyBuilder AddEvilDeed(string deed) { _enemy.EvilDeeds.Add(deed); return
this; }
```

```
    public Character Build() { return _enemy; }
}

#endregion

#region Character class

public class Character
{
    public int Height { get; set; }
    public string Build { get; set; } = "";
    public string HairColor { get; set; } = "";
    public string EyeColor { get; set; } = "";
    public string Clothes { get; set; } = "";
    public List<string> Inventory { get; set; } = new List<string>();

    public List<string> GoodDeeds { get; set; } = new List<string>();
    public List<string> EvilDeeds { get; set; } = new List<string>();

    public void Print()
    {
        Console.WriteLine($"Height: {Height}, Build: {Build}, Hair: {HairColor}, Eyes:
{EyeColor}, Clothes: {Clothes}");
        Console.WriteLine($"Inventory: {string.Join(", ", Inventory)}");
        if (GoodDeeds.Count > 0) Console.WriteLine($"Good Deeds: {string.Join(", ",
GoodDeeds)}");
        if (EvilDeeds.Count > 0) Console.WriteLine($"Evil Deeds: {string.Join(", ",
EvilDeeds)}");
        Console.WriteLine(new string('-', 40));
    }
}

#endregion

#region Director

public class CharacterDirector
{
    public void CreateHero(HeroBuilder builder)
    {
        builder.SetHeight(180)
                .SetBuild("Athletic")
                .SetHairColor("Blonde")
                .SetEyeColor("Blue")
                .SetClothes("Knight Armor")
                .AddInventory("Sword")
                .AddInventory("Shield")
                .AddGoodDeed("Saved the village")
                .AddGoodDeed("Defeated the dragon");
    }

    public void CreateEnemy(EnemyBuilder builder)
    {
        builder.SetHeight(190)
                .SetBuild("Muscular")
                .SetHairColor("Black")
                .SetEyeColor("Red")
                .SetClothes("Dark Robes")
                .AddInventory("Dark Sword")
                .AddInventory("Poison")
                .AddEvilDeed("Burned the village")
                .AddEvilDeed("Kidnapped the princess");
    }
}
```

```
#endregion

class Program
{
    static void Main(string[] args)
    {
        var director = new CharacterDirector();

        var heroBuilder = new HeroBuilder();
        director.CreateHero(heroBuilder);
        var hero = heroBuilder.Build();

        var enemyBuilder = new EnemyBuilder();
        director.CreateEnemy(enemyBuilder);
        var enemy = enemyBuilder.Build();

        Console.WriteLine("Hero:");
        hero.Print();

        Console.WriteLine("Enemy:");
        enemy.Print();
    }
}
```

5. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Результат виконання програми:

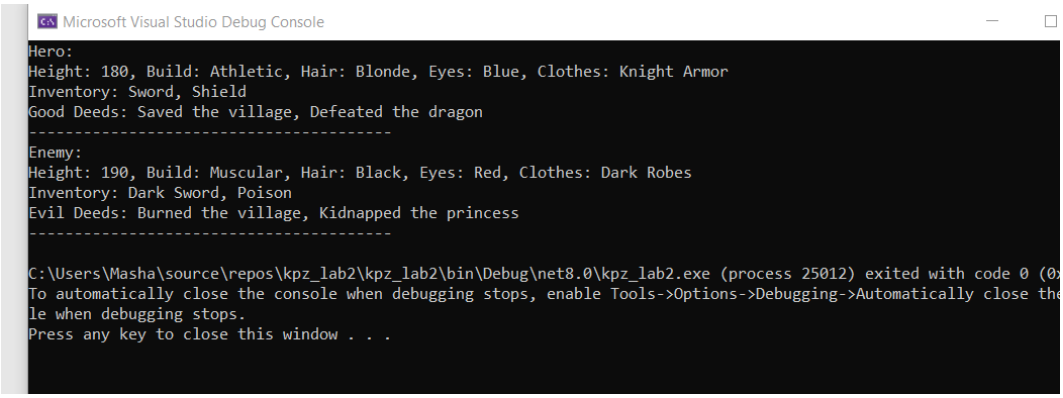

Рис.7. Результат виконання програми.

6. Підготуйте діаграму створених у програмі класів та інтерфейсів за допомогою https://app.diagrams.net/, експортуйте та завантажте її до репозиторія.

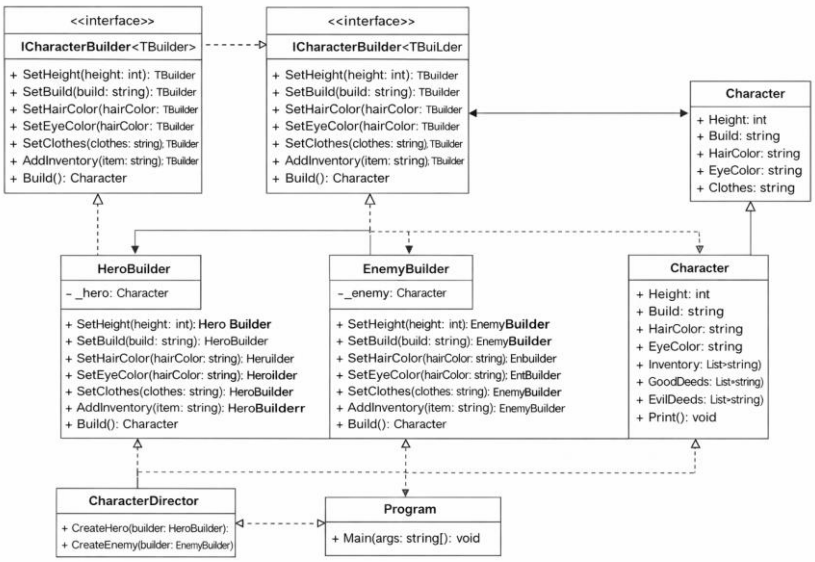

Рис.8. Діаграма створених у програмі класів.

Висновок: навчилася реалізовувати породжувальні шаблони проєктування.